

Motivation and aim

RSA security relies on the primes used to build each modulus $N = p \cdot q$ being unpredictable and never reused. If two moduli share a prime factor, computing $\gcd(N_i, N_j)$ is enough to compromise both private keys.

This scenario is especially relevant for digital certificates, because the RSA public key is embedded in the certificate and can be collected openly. *Certificate Transparency* (CT) logs therefore provide a large public source of real Internet keys for cryptographic auditing.

Unlike broad Internet scans, CT logs mainly contain certificates accepted by the public Web PKI and logged for transparency. This makes them a useful empirical source for studying RSA keys that are actually deployed in the browser-trusted ecosystem, rather than only in test, internal, or self-signed environments.

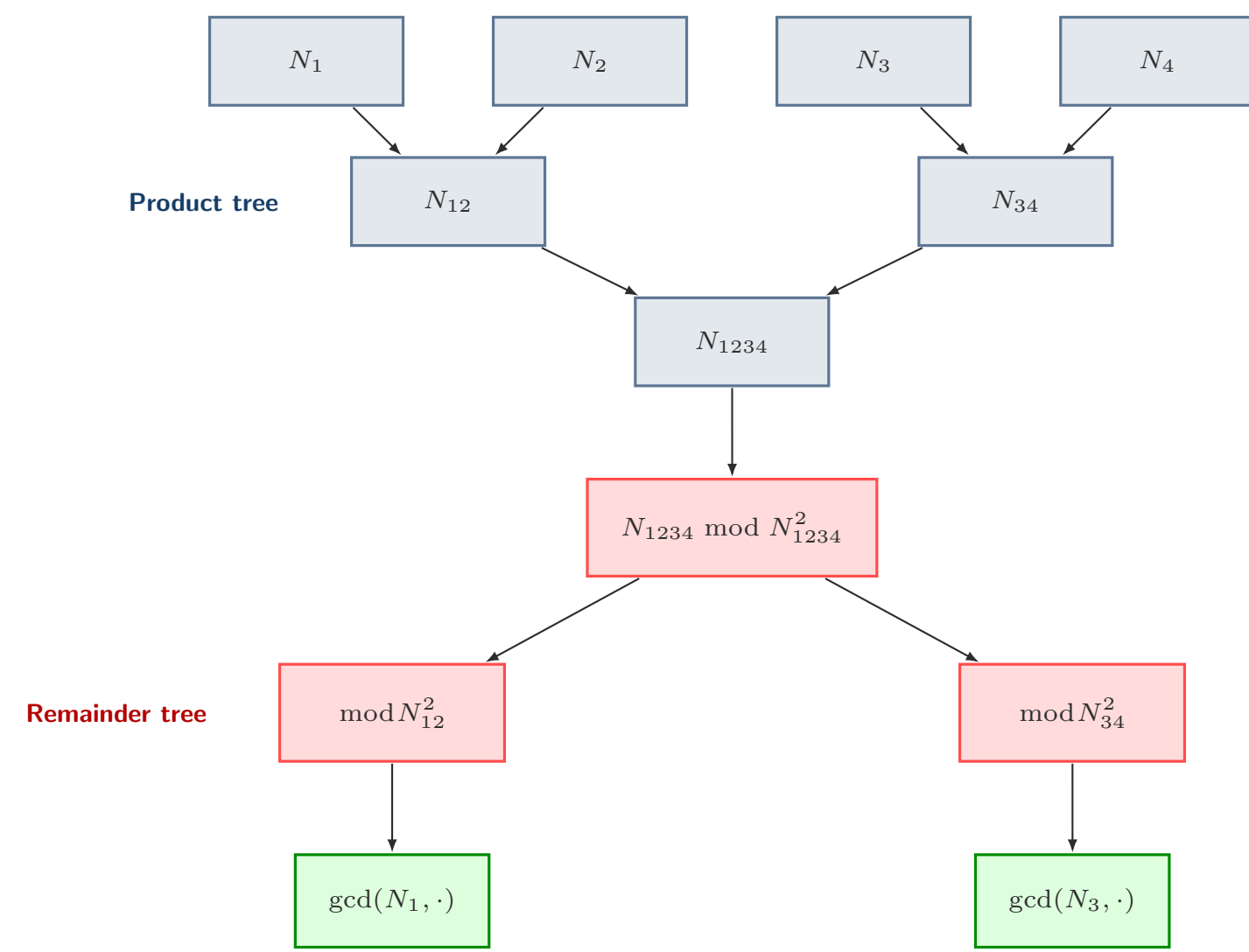
The problem was previously studied in the master's thesis *Finding shared RSA factors in the Certificate Transparency logs*, which analyzes real certificates extracted from CT logs with classical *batch GCD* techniques. That work shows the practical value of auditing CT logs, but also the high computational cost of large-scale analysis.

As an algorithmic improvement, Pelofske proposes *Binary Tree Batch GCD*, a more efficient method that avoids part of the cost of the classical *remainder tree* approach.

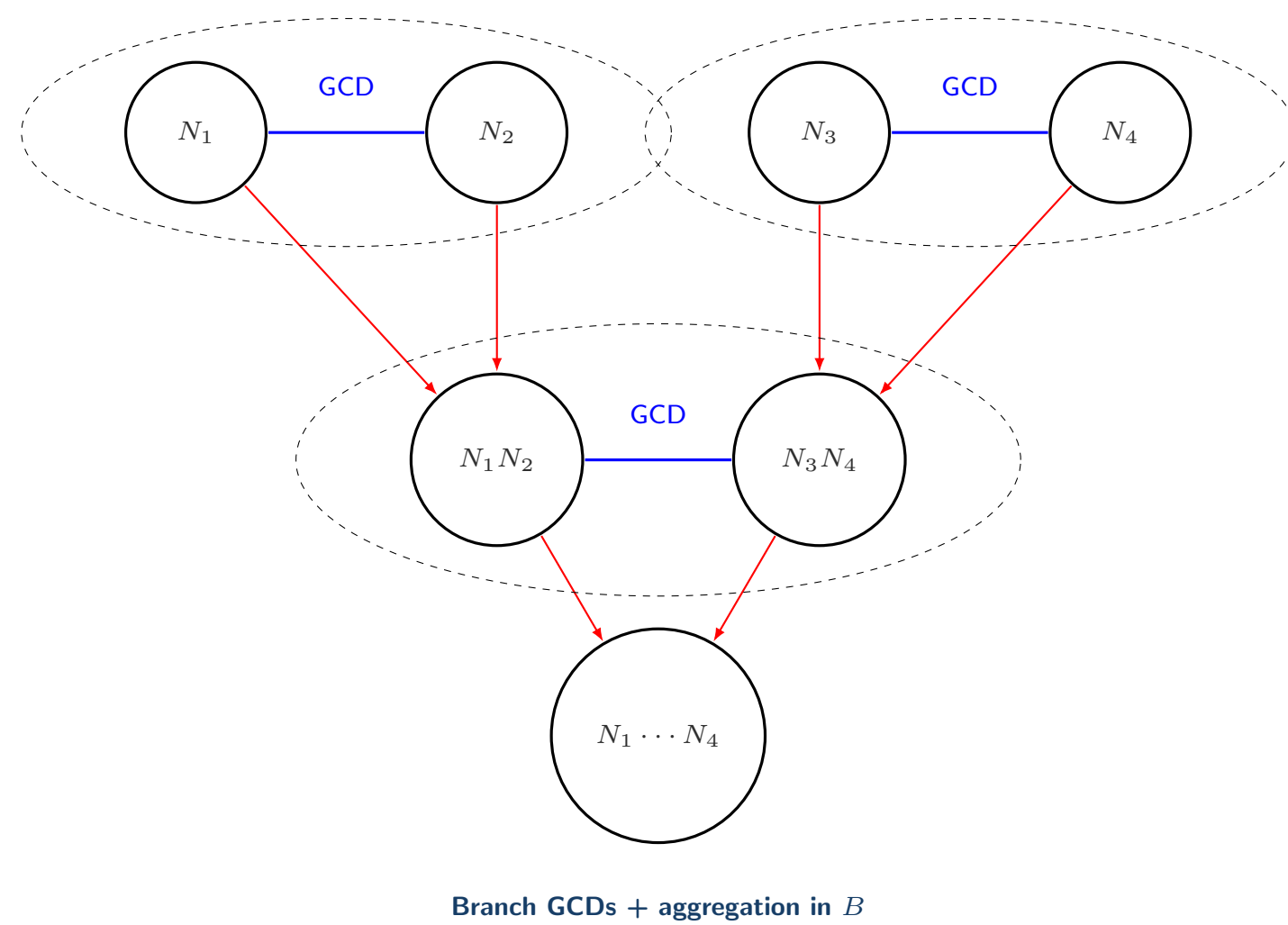
This thesis connects both lines of work: it searches for shared RSA factors in CT logs, evaluates *Binary Tree Batch GCD* against the classical method, and develops a C++17/GMP/OpenMP implementation suitable for large-scale runs.

Algorithm and implementation

The naive approach requires $\mathcal{O}(M^2)$ GCD computations for M RSA moduli. Classical *batch GCD* avoids pairwise comparison through a *product tree* and a *remainder tree*, but the second structure is expensive in time and memory.



Binary Tree Batch GCD removes the full *remainder tree*. During product-tree construction it computes GCDs between sibling branches, stores the non-trivial GCDs in a single aggregate value B , and finally checks $\gcd(N_i, B)$ for each input modulus. This is efficient when shared factors are rare compared with the total number of keys, as expected in real CT logs.

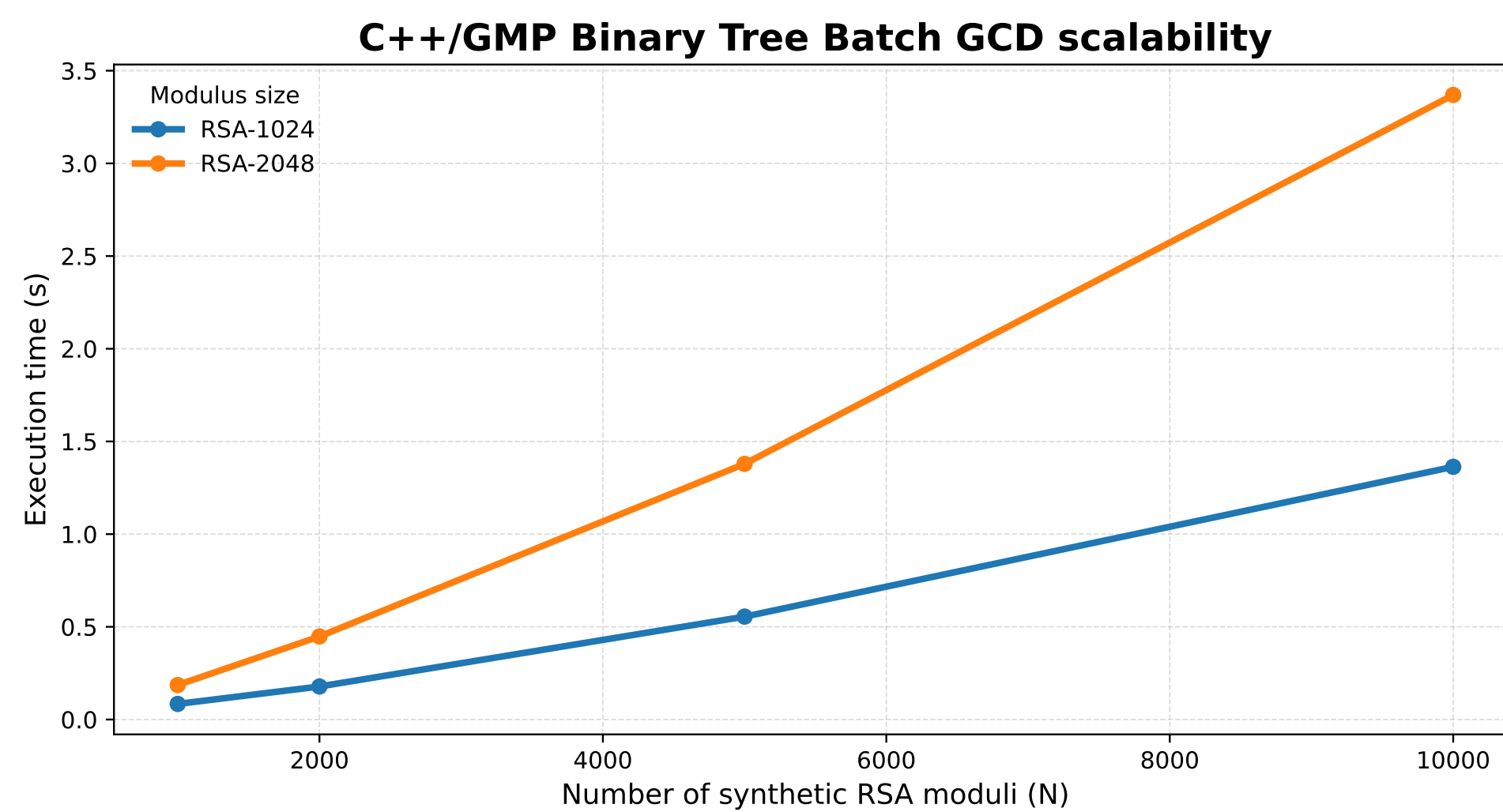


The implementation follows three phases: product-tree construction with in-line GCD computation, aggregation of non-trivial GCDs into B , and final enumeration of the original moduli through $\gcd(N_i, B)$. Intermediate products are released as soon as they are no longer needed, and the final enumeration is parallelized with OpenMP.

Pelofske's Python versions are used as references, but Python overhead on large integers limits scalability. The final implementation therefore uses C++17 with GMP and OpenMP to improve multiprecision arithmetic, memory control, and parallel enumeration.

Experimental design

Validation combines two phases. First, synthetic experiments use RSA moduli of 1024 and 2048 bits with a controlled number of weak keys. This checks that the algorithm detects the inserted shared factors and measures execution time as the input size grows.

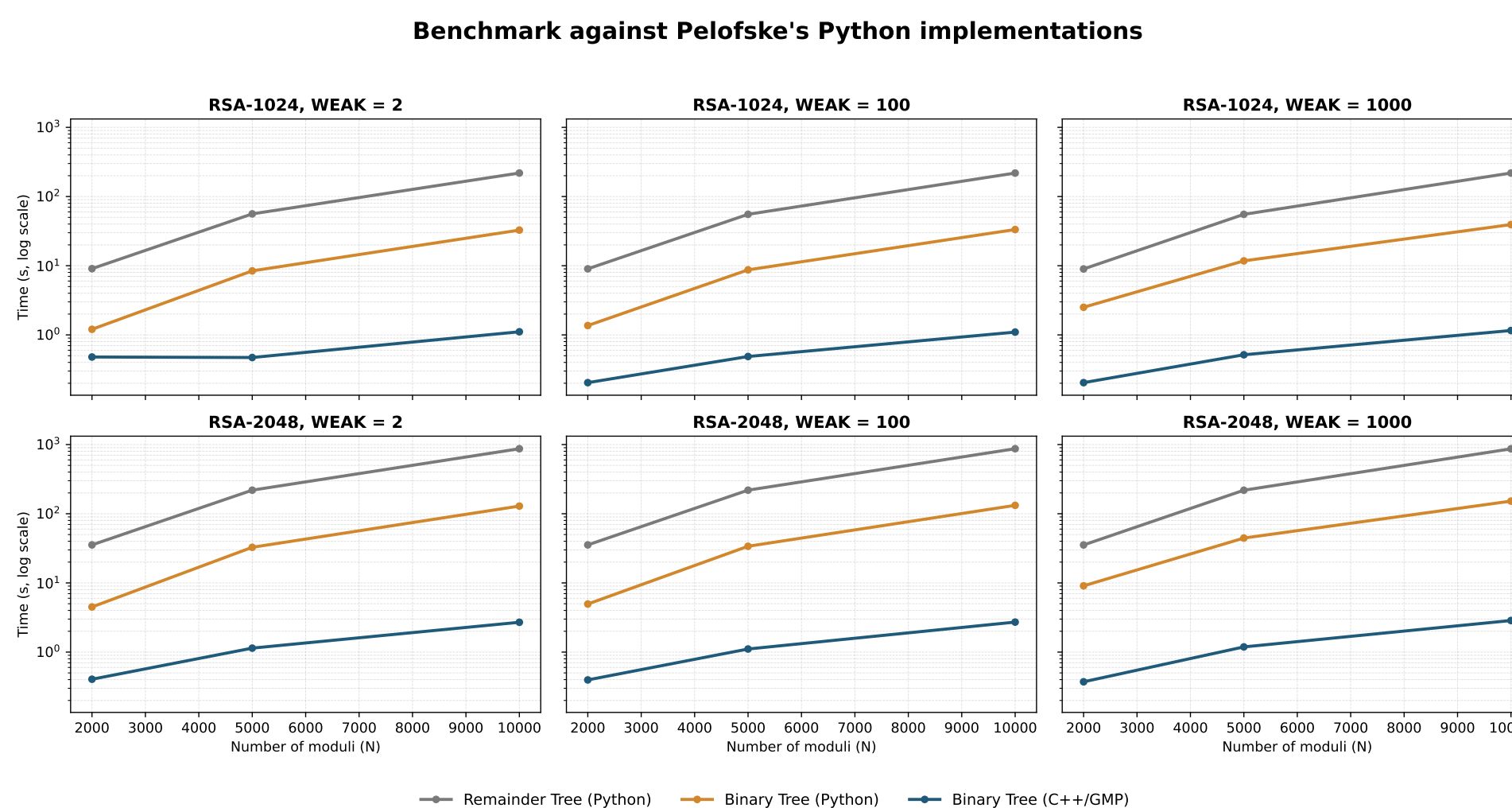


Second, the repository associated with Pelofske's article is used to reproduce a comparison close to the original experiment. The benchmark evaluates three variants on the same inputs: classical *Remainder Tree Batch GCD* in Python, *Binary Tree Batch GCD* in Python, and the C++/GMP implementation developed in this thesis.

For each configuration, synthetic RSA moduli are written as hexadecimal values, one modulus per line. The same input file is then used by all three algorithms, so the comparison reflects the algorithmic and implementation cost rather than differences in the generated data.

Integrating the reference repository required adapting the code to current Python and NumPy versions, adding execution wrappers, normalizing module input, and generating the additional primes required for RSA-2048 tests.

Benchmark results



The extended benchmark contains 18 completed configurations combining 1024- and 2048-bit RSA moduli, three input sizes, and three values of WEAK. Across those configurations, Python *Binary Tree Batch GCD* is on average $6.1\times$ faster than Python *Remainder Tree Batch GCD*. The C++/GMP implementation is on average $156.9\times$ faster than Python *Remainder Tree Batch GCD* and $26.0\times$ faster than Python *Binary Tree Batch GCD*. The vertical axis is logarithmic because the runtime gap between Python and C++/GMP is too large for a linear comparison. The same synthetic input was used by the three implementations and the detected vulnerable moduli matched, so the figure compares execution cost rather than differences in correctness. The pattern is consistent: increasing N dominates runtime, RSA-2048 is slower than RSA-1024, and avoiding the explicit *remainder tree* is always beneficial.

Observed speed-up

The most demanding local benchmark configuration uses RSA-2048 moduli, $N = 10000$ and WEAK=1000: the classical Python method takes 867.508 s, Pelofske's Python binary-tree version takes 152.171 s, and the C++/GMP implementation takes 2.860 s.

Bits	WEAK	N	Remainder Py.	Binary Py.	Binary C++
1024	100	10000	218.468 s	33.337 s	1.097 s
2048	100	10000	870.560 s	132.189 s	2.714 s
2048	1000	10000	867.508 s	152.171 s	2.860 s

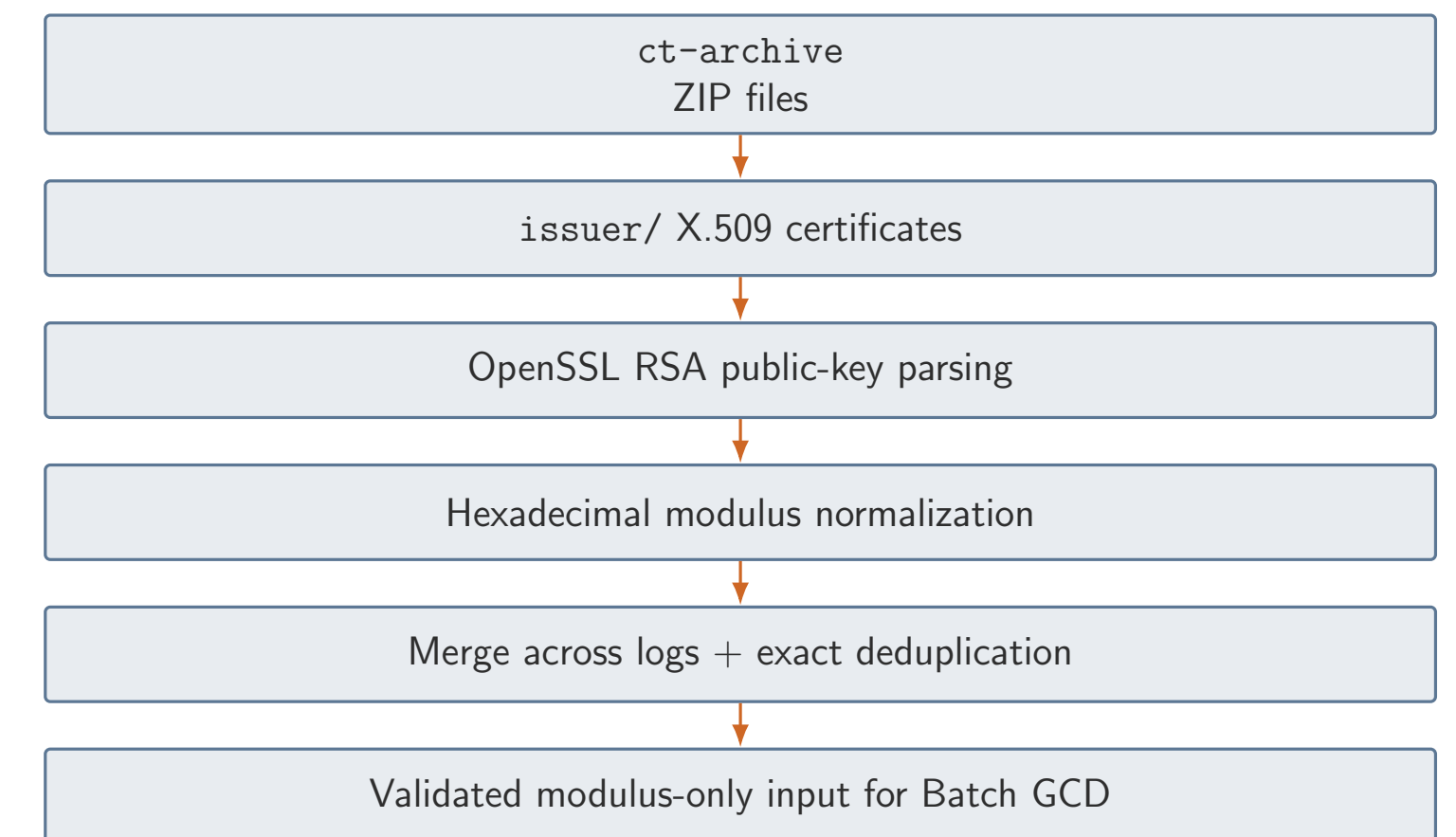
C++/GMP reaches up to $322.4\times$ speed-up over the classical Python method.

In practical terms, the hardest displayed case moves from more than 14 minutes with the classical Python baseline to under 3 seconds with the C++/GMP binary. That margin makes repeated validation, archive corrections, and real-data reruns feasible within the same CT-log audit workflow.

Real RSA Modulus Extraction

The real-data pipeline starts from selected public ct-archive ZIP files. The workflow is incremental: one archive is downloaded, inspected and processed independently, then only the extracted metadata needed for the audit is kept. This avoids storing full certificate collections permanently and makes large or problematic ZIP files easier to retry. The extraction state of each ZIP is tracked explicitly so interrupted downloads, Zip64 issues, and archives without issuer/ entries can be treated conservatively.

Only X.509 issuer certificates under the issuer/ directory are parsed. OpenSSL extracts both the RSA public-key modulus and the certificate issuer. The intermediate output keeps pairs of the form modulus|issuer; the issuer is not needed for Batch GCD, but it would provide context if a vulnerable modulus were found.



- Processing issuer/ avoids traversing the full CT tile data and keeps the extraction feasible ZIP by ZIP.
- Each archive is recorded as processed, failed, or apparently without issuer/; suspicious cases are reviewed before being discarded.
- The Modulus= prefix and formatting artifacts are removed, so each output line contains exactly one hexadecimal integer.
- Duplicates caused by renewals, equivalent certificates, or repeated appearances across logs are removed before the GCD computation.
- Plausibility validation checks the modulus-only input before the C++/GMP binary runs.

Final CT-Log Audit

The final audit uses this clean modulus-only input after normalization, exact deduplication, and validation of plausible RSA moduli. The validation step removes malformed values and checks that the remaining integers behave like RSA moduli suitable for the attack input.

Metric	Value
Raw unique RSA moduli	54,278
Plausible RSA moduli analyzed	54,275
Moduli discarded during validation	3
CT logs with extracted moduli	20
Incomplete logs at execution time	0
Non-trivial GCDs in phase 1	0
Vulnerable keys detected	0
Total algorithm time	23.756 s

The final run did not detect shared RSA factors in the analyzed data set. This does not prove that all CT-log certificates are safe; it states that no shared factor was found in the final deduplicated set processed by this thesis. The result is therefore an applied audit result for this extracted data set, not a global claim about every certificate ever logged in CT.

Conclusions

- Binary Tree Batch GCD* reduces the practical cost of shared factor detection compared with the classical *remainder tree* method.
- The C++17/GMP implementation clearly outperforms the original Python implementations on the reproduced benchmark.
- The complete extraction, normalization, validation, and analysis flow was run on real CT-log data.
- No vulnerable RSA key was detected among the 54,275 plausible unique RSA moduli analyzed.

The main contribution is a reproducible end-to-end workflow that links CT-log data engineering with an efficient shared-factor audit for real RSA public keys.

References

- R. L. Rivest, A. Shamir, L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems", *Communications of the ACM*, 1978.
- N. Heninger et al., "Mining Your Ps and Qs", *USENIX Security*, 2012.
- H. F. Vâge, "Finding shared RSA factors in the Certificate Transparency logs", Master's Thesis, University of Bergen, 2022.
- E. Pelofske, "An Efficient All-to-All GCD Algorithm for Low Entropy RSA Key Factorization", Preprint, 2024.